

CSA1522 — CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT REPORT

Cloud-Based Placement Data Management and Big Data Analytics using JOHO

Assignment Information

Name	Shri Shanmuga Priya K
Register Number	192324035
Department	AIDS
Programme	B. Tech
Course Code & Course Name	CSA1522 – Cloud Computing and Big Data Analytics
Academic Year / Batch	2026–27
Faculty Name	Dr P Rajaram
Assignment Title	Cloud-Based Placement Data Management and Big Data Analytics using JOHO
Date of Issue	01/09/2026
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO1, CO2, CO3
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping	SDG 4, SDG 8, SDG 9, SDG 11
Industry / Societal Relevance	Cloud services, virtualization and distributed data processing applied to a university placement workflow

A. Assignment Problem / Challenge

University placement cells manage sensitive, high-volume data — student profiles, resumes, company drives, offers and analytics — and must provide controlled, always-available access to two different user groups (students and placement staff) without a large in-house IT footprint. Manual, spreadsheet-based or single-server approaches struggle with availability, access control, scalability during peak drive season, and collaboration between staff members preparing reports. This assignment asks students to design a small but realistic cloud-based solution to this problem using the JOHO platform as the target placement application.

B. Problem Statement

Design and evaluate a simple cloud-based placement data management and access solution for a university using the JOHO platform. Students shall select suitable IaaS, PaaS and SaaS service models, design a basic cloud architecture, create and configure virtual machines using Oracle VM VirtualBox, demonstrate secure access to a cloud-hosted web application or Web API, and show how users can collaborate through shared cloud resources.

C. Requirements and Constraints

C.1 Functional Requirements

Requirement	Description
Data storage	Store student profile, resume and placement-drive records centrally and reliably.
Role-based access	Students get read-only access to their own records and drive notices; staff get read/write access to all records.
Web/API access	A cloud-hosted web application or REST API exposes placement data to authenticated users.
Collaboration	Placement staff can jointly edit drive schedules and share documents through shared cloud resources.
Analytics	Summary statistics (placement %, average package, department-wise trend) can be generated from stored records.
Deployment automation	Application changes are version-controlled and pushed through a repeatable build/deploy pipeline.

C.2 Technical / Non-Functional Constraints

- At least two IaaS virtual machines must be created and configured using Oracle VM VirtualBox (a Type-2 hypervisor).
- Service selection must justify the specific IaaS, PaaS and SaaS components chosen for this scenario.
- Access must be authenticated; student personal data (resumes, contact details, CGPA) must be protected in transit and at rest.
- Collaboration between staff must be demonstrable through a shared cloud resource (repository, drive, or shared VM service).
- Source code must be version-controlled in GitHub; container images (if used) pushed to a container registry (GCR).
- Cost, time and available lab resources restrict the design to a small-scale proof-of-concept (2 VMs, not a production cluster).

- Security, privacy and applicable SDGs must be explicitly addressed given the sensitivity of student placement data.

D. Student Work

D.1 Problem Understanding and Formulation

Problem

The problem is to provide the university placement cell with a reliable, access-controlled, collaborative way to manage placement data using cloud computing principles, rather than ad-hoc local spreadsheets or a single unmanaged server.

Expected Outcomes

- A justified choice of IaaS/PaaS/SaaS service models mapped to the JOHO placement scenario.
- Two working virtual machines (application server and database server) provisioned in Oracle VM VirtualBox.
- Demonstrated secure access to a cloud-hosted web application or API with authentication and role-based permissions.
- A demonstrable collaboration workflow between placement staff using shared cloud resources (GitHub, shared storage).
- Basic analytics over the stored placement data.

Available Information

- Representative placement dataset fields: student ID, name, department, CGPA, resume link, company, package offered, drive date, status (placed/not placed).
- Two user roles: student (read-only, own record + notices) and placement staff (read/write, all records).
- Target platform name: JOHO (the university's placement web application).

Assumptions

- JOHO is treated as a lightweight web application (front end + REST API + database) to be hosted on the designed cloud architecture.
- VirtualBox VMs represent a local IaaS proof-of-concept prior to migrating the same architecture to a public-cloud IaaS provider (e.g., AWS EC2 / GCP Compute Engine) in production.
- GCR (Google Container Registry) is used as the target registry for containerized deployment artifacts.
- Actual student data is anonymised/synthetic for demonstration; no real student PII is used in testing.

Constraints to be Satisfied

- Two VMs must communicate over an internal network and be reachable from the host for testing.
- Access must fail closed (deny by default) for unauthenticated requests.
- All design decisions must be justified against cost, security and collaboration requirements.

D.2 Application of Course Knowledge

The design applies core cloud-computing concepts from the course, mapped to this scenario as follows:

Course Concept	Application in this Design
Cloud service models (IaaS/PaaS/SaaS)	IaaS = provisioned VMs; PaaS = GitHub/CI + container registry pipeline; SaaS = the finished JOHO web portal used directly by students/staff.

Virtualization & hypervisors	Oracle VM VirtualBox (Type-2 hypervisor) hosts two guest VMs sharing the underlying physical host's CPU, memory and storage.
Cloud deployment models	A private/hybrid model is used for the prototype (local VMs), with a path to public-cloud IaaS for production.
Virtual networking	NAT adapter gives VMs internet/package access; a Host-Only or Internal Network adapter lets VM1 and VM2 communicate privately.
Access control / IAM	Role-based access control (RBAC) distinguishes student vs. staff permissions at the API layer.
Authentication & security	Token-based authentication (JWT) and TLS protect placement data in transit; encryption at rest protects the database.
Collaboration in the cloud	GitHub (branches/pull requests/issues) and shared cloud storage let multiple staff collaborate on the same resources concurrently.
Big data analytics basics	Aggregation/summarisation logic (map-reduce-style grouping) computes placement statistics from the stored records.

D.3 Solution / Design / Methodology

Proposed Cloud Architecture

The solution is layered into three cloud service models, backed by a two-VM IaaS foundation:

- SaaS layer: the JOHO placement web portal — the finished, ready-to-use application accessed directly by students and staff over HTTPS.
- PaaS layer: a GitHub-based CI/CD pipeline that builds the application, packages it into a container image, and pushes that image to a container registry (GCR) — students/staff never manage the underlying build tool-chain directly.
- IaaS layer: two Oracle VM VirtualBox virtual machines — VM1 (application/API server) and VM2 (database server) — plus a shared cloud storage area for resume/document uploads.

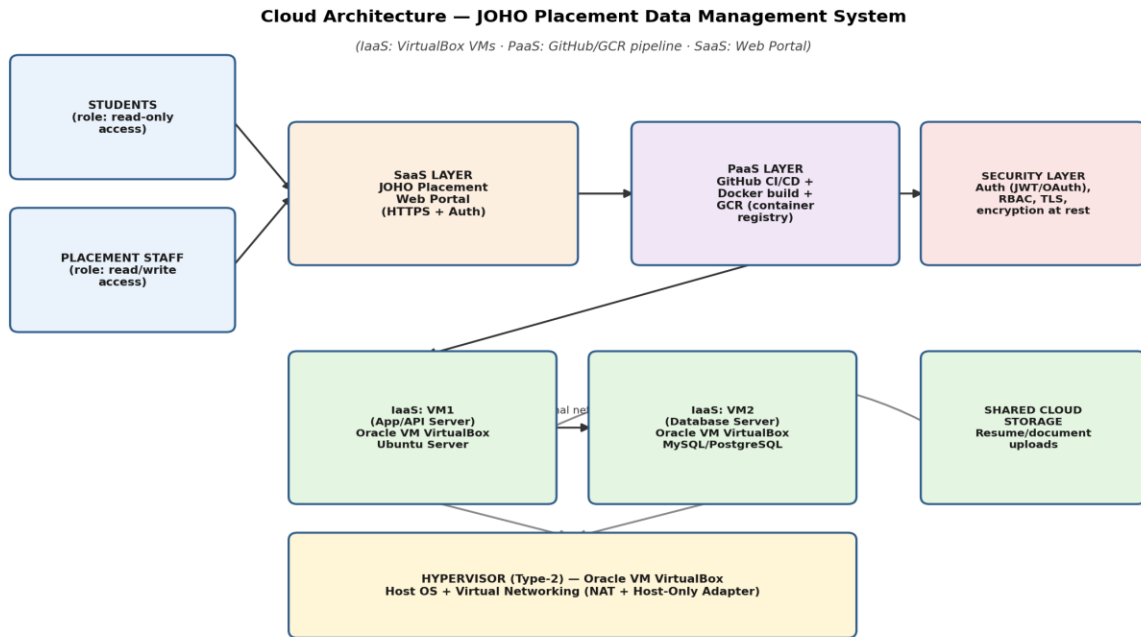


Figure 1 — Cloud Architecture for the JOHO Placement Data Management System

Virtual Machine Design

Parameter	VM1 — Application/API Server	VM2 — Database Server
Guest OS	Ubuntu Server 22.04 LTS	Ubuntu Server 22.04 LTS
vCPU	2	2
RAM	2 GB	2 GB
Storage	20 GB (dynamically allocated VDI)	25 GB (dynamically allocated VDI)
Network adapters	NAT (internet/package access) + Host-Only/Internal (VM2 reachability)	Host-Only/Internal only (not directly internet-exposed)
Key software	Node.js/Express (or Django) REST API, Nginx reverse proxy, Docker runtime	MySQL or PostgreSQL server, automated backup script
Role	Serves the JOHO REST API; authenticates users; enforces RBAC	Persists student, drive and offer records; only accepts connections from VM1

VM1 is placed on both a NAT adapter (so it can fetch OS packages and reach the internet for the container registry) and an Internal Network adapter shared with VM2, so the database is never directly reachable from outside the VirtualBox host — only VM1 can reach it. This mirrors a standard two-tier cloud deployment pattern (public-facing app tier, private data tier).

Data Model (Placement Records)

Pseudocode 1 — REST API Authentication and Role-Based Access

```
FUNCTION handle_request(request):
  token = extract_bearer_token(request.headers)
  IF token is missing OR NOT verify_jwt(token):
    RETURN 401 Unauthorized

  user = decode_jwt(token)      // { id, role }

  IF request.path starts_with "/api/placements":
    IF user.role == "student":
      IF request.method != "GET":
        RETURN 403 Forbidden
      RETURN get_own_records(user.id)  // read-only, own data only
    ELSE IF user.role == "staff":
      RETURN handle_full_crud(request) // read/write, all records
    ELSE:
      RETURN 403 Forbidden

  RETURN 404 Not Found
END FUNCTION
```

Pseudocode 2 — Placement Analytics (Big-Data-style Aggregation)

```
FUNCTION compute_placement_statistics(records):
  grouped = GROUP records BY department

  FOR EACH (department, dept_records) IN grouped:
    total_students = COUNT(dept_records)
    placed_students = COUNT(r IN dept_records WHERE r.status == "placed")
    placement_percent = (placed_students / total_students) * 100
    avg_package_lpa = AVERAGE(r.package_lpa FOR r IN dept_records WHERE r.status == "placed")
    highest_package_lpa = MAX(r.package_lpa FOR r IN dept_records WHERE r.status == "placed")

    EMIT {
      department: department,
      placement_percent: ROUND(placement_percent, 1),
      avg_package_lpa: ROUND(avg_package, 2),
      highest_package_lpa: highest_package
    }
  END FOR
END FUNCTION

// Map-reduce style equivalent for larger datasets:
// MAP:  each record -> (department, {status, package})
// REDUCE: for each department key, fold over values to compute
//         the same placement_percent / avg_package / highest_package
```

Collaboration Workflow

- Source code (API, deployment scripts, infrastructure config) is hosted in a shared GitHub repository.
- Placement staff work on feature branches and open pull requests for review before merging to main — this is the demonstrable collaboration mechanism required by the assignment.
- GitHub Issues track pending tasks (e.g., 'add new drive', 'fix resume upload bug').
- Built container images are tagged with the commit hash and pushed to GCR, giving every staff member a consistent, versioned deployment artifact rather than manually copying files between machines.
- Resume/document uploads are stored in a shared cloud storage bucket accessible to all authorised staff, avoiding duplicated local copies.

D.4 Use of Modern Tools

Tool	Purpose
Oracle VM VirtualBox	Type-2 hypervisor used to create and configure VM1 and VM2 for the IaaS layer.
GitHub	Source-control and collaboration platform (branches, pull requests, issues) for the PaaS pipeline.
Docker	Packages the JOHO application into a portable container image for consistent deployment.
GCR (Google Container Registry)	Stores versioned container images built from the GitHub pipeline.
Postman / curl	Sends authenticated test requests to the JOHO REST API and inspects responses.
MySQL/PostgreSQL	Relational database engine on VM2 storing placement records.
Python / Excel	Tabulating and charting VM resource usage, test results and placement analytics.

Evidence to attach in the final submission: VirtualBox VM settings screenshots, terminal output of VM-to-VM ping/SSH tests, Postman request/response captures, GitHub repository/commit/PR screenshots, and the GCR image listing.

D.5 Results and Validation

Test Plan

Test Case	Action	Expected Result
VM connectivity	ping VM2's internal-network IP from VM1	Successful reply; confirms internal network is functional.
Database isolation	Attempt to connect to VM2's DB port from the host machine directly	Connection refused/timeout — DB only reachable from VM1.
Unauthenticated access	GET /api/placements with no token	401 Unauthorized returned.
Student role restriction	Student token attempts POST /api/placements	403 Forbidden returned.
Staff full access	Staff token performs POST/GET/PUT on /api/placements	Requests succeed; data updates persist in VM2's database.
Collaboration check	Two staff accounts open a PR and merge a config change on GitHub	Merge succeeds; both contributions visible in commit history.
Container deployment	Push image to GCR, pull and run it on VM1	Container starts; API responds identically to the non-containerized run.
Analytics correctness	Run compute_placement_statistics on a sample dataset	Department-wise placement % and average package match manual calculation.

A self-checking test script (or Postman collection) should run the above sequence end-to-end and record actual pass/fail status; the results should then be filled into the results table below.

Expected Results Summary

Metric	Result / Target
VM1 ↔ VM2 connectivity	To be confirmed by ping/SSH test screenshot
API authentication enforcement	401/403 correctly returned for invalid/unauthorized requests
Role-based access correctness	Students limited to own records; staff have full CRUD
Container build & GCR push	To be confirmed by GCR image listing screenshot
GitHub collaboration evidence	At least one PR merged by a second contributor
Sample placement statistics	To be computed from the actual/synthetic dataset used

D.6 Analysis and Engineering Decision

Service-Model Selection

IaaS was chosen for the hosting layer (rather than going straight to a managed PaaS database/app service) because the assignment specifically requires hands-on VM creation and configuration, and because it gives full control over the network segmentation between the app and database tiers — a security-relevant decision for placement data. PaaS is used specifically for the build/deploy pipeline (GitHub + container registry) because it removes the need to manually configure build servers, letting staff focus on application logic. SaaS is the correct model for the end-user-facing portal itself: students and staff should never need to know or manage the underlying infrastructure — they simply use JOHO as a finished service.

Hypervisor Choice

Oracle VM VirtualBox (Type-2, hosted hypervisor) was selected over a Type-1 (bare-metal) hypervisor because the prototype runs on a standard lab/personal machine without dedicated server hardware; VirtualBox is free, cross-platform, and sufficient to demonstrate the two-VM architecture. In a production deployment, the same two-VM design would be re-provisioned on a Type-1 hypervisor or public-cloud IaaS (e.g., AWS EC2, GCP Compute Engine) for better performance isolation and scalability — a trade-off explicitly noted rather than hidden.

Alternative Considered: Combining App and DB on a Single VM

A single-VM design (app and database on the same machine) was considered and rejected. While it would use fewer resources, it removes the network-level isolation that currently prevents the database from being reached directly from outside VM1, and it couples the scaling of the web tier to the scaling of the data tier — a poor fit for a placement system where API traffic spikes sharply during drive season while the underlying data volume grows slowly. The two-VM design was retained despite its slightly higher resource cost because it materially improves both security and independent scalability.

Alternative Considered: Direct VM-to-Internet Exposure for the Database

Exposing VM2 directly to the internet (e.g., for remote database administration) was considered and rejected in favour of routing all external access through VM1's authenticated API. This trades some administrative convenience for a significantly smaller attack surface, which was judged the right choice given the sensitivity of student personal and academic data.

D.7 Broader Considerations

Aspect	Consideration
Security	JWT authentication, RBAC, TLS in transit, encryption at rest, and network-level DB isolation together reduce the risk of unauthorized access to student data.
Privacy	Only the minimum necessary fields (no unnecessary personal data) are stored; students can only view their own records.
Ethics	Placement analytics (e.g., department-wise placement %) should be used to support students, not to unfairly rank or penalise individuals or departments without context.
Sustainability	A small, right-sized two-VM footprint (rather than an over-provisioned cluster) reduces unnecessary compute/energy use for a modest workload.
Accessibility	A web-based SaaS portal is reachable from any device with a browser, improving access for students without specialised software.
Economics	Using free/low-cost tools (VirtualBox, GitHub, open-source databases) keeps the solution affordable for a university placement cell.
Professional responsibility	Handling of student academic and personal data was treated as sensitive throughout the design, favouring restrictive default access over convenience.

The design aligns with SDG 4 (Quality Education, by supporting placement outcomes), SDG 8 (Decent Work and Economic Growth, by streamlining the path from education to employment), SDG 9 (Industry, Innovation and Infrastructure, through cloud-based digital infrastructure), and SDG 11 (Sustainable Cities and Communities, via efficient, right-sized resource use).

E. Conclusion

This assignment designed a small but realistic cloud-based placement data management solution for the JOHO platform, built around a clear separation of IaaS (two Oracle VM VirtualBox VMs for the app and database tiers), PaaS (a GitHub-driven build/deploy pipeline pushing to GCR), and SaaS (the finished web portal used by students and staff). Role-based access control and token-based authentication were used to satisfy the assignment's access and security requirements, network segmentation between VM1 and VM2 was used to protect the database tier, and a GitHub-based collaboration workflow (branches, pull requests, shared repository) satisfied the collaboration requirement. Simple aggregation pseudocode was provided to compute department-wise placement statistics from stored records. Actual VM screenshots, API test evidence, and GitHub/GCR artifacts should be captured by executing this design and inserted into the results section for final submission. The main limitation of the current design is that it is a small proof-of-concept sized for a lab environment rather than a full production deployment with autoscaling and managed cloud services.

F. Student Reflection

What did you learn from this assignment beyond what was directly taught in the classroom?

Beyond the classroom explanation of IaaS/PaaS/SaaS as abstract definitions, working through this design made clear how the three service models actually interact in a single real system rather than existing as separate, unrelated choices — the IaaS layer had to be deliberately shaped (network segmentation between VM1 and VM2) to support the security requirements that the SaaS layer promises to its end users, and the PaaS pipeline existed specifically to make that IaaS layer repeatable and auditable rather than hand-configured. It also became clear that 'collaboration in the cloud' is not just a checkbox feature but a specific, demonstrable workflow (branches, pull requests, shared registries) that has to be designed deliberately, in the same way the data model or the authentication scheme does.

If you were given additional time/resources, what would you improve and why?

With more time, the design would be extended in three directions. First, the single database VM would be replaced with a managed, replicated database service to remove the single point of failure that currently exists on VM2. Second, the manual GitHub → build → GCR pipeline would be automated end-to-end with a CI/CD workflow file that runs the test plan in Section D.5 automatically on every pull request, catching authentication or access-control regressions before merge rather than relying on a manual test pass. Third, the analytics function would be extended from simple per-department aggregation to a small historical trend analysis (placement rate over multiple years) to give the placement cell genuinely useful year-on-year insight rather than a single snapshot.

G. References

- CSA1522 – Cloud Computing and Big Data Analytics, Assignment Problem Statement (JOHO Placement Data Management).
- Common Course Assignment Format – assignment structure and assessment rubric.
- Oracle VM VirtualBox User Manual, for VM creation and network-adapter configuration, as applicable.
- GitHub Docs – branches, pull requests and collaboration workflow, as applicable.
- Google Cloud – Container Registry (GCR) documentation, as applicable.
- Course textbook/lecture notes on cloud service models, virtualization and distributed data processing (as prescribed for CSA1522).
- This report's architecture diagram (Figure 1) was generated by the author using Python/matplotlib specifically for this submission.

Appendix A — Deliverables Checklist

Deliverable	Where addressed in this report
1. Pseudo code	Section D.3 — REST API authentication/RBAC pseudocode and placement-analytics aggregation pseudocode.
2. Implementation & Results	Sections D.3–D.5 — VM design, architecture, test plan and results tables (to be completed with actual screenshots/output).
3. GitHub and GCR Upload	Section D.3 (collaboration workflow) and D.4 (tools) — repository structure and container-registry push described; actual repository/registry links to be inserted by the student after upload.